

ExpressJS Notes

[- Mo Fahim Raj](#)

Express JS.....	4
What is Express.js?.....	4
Why Use Express.js?.....	5
How to Set Up Express.js.....	5
Create a New Project Folder.....	5
Initialize Node.js Project.....	5
Install Express.....	5
Create Entry File – index.js.....	5
Basic Express Server.....	5
Auto Restart with Nodemon.....	6
Install Nodemon.....	6
How Express JS Works Explained.....	6
Understand Express.js Flow.....	6
Require Express.....	6
Start the Server.....	7
Check app Object.....	7
Express.js Routes and Flow.....	7
Basic Route Example.....	7
Multiple Routes Example.....	7
Module Type in package.json Explained.....	8
Why ES Modules?.....	8
Enable ES Module Support.....	8
Before (CommonJS Syntax).....	8
After (ES Module Syntax).....	8
Create Pages Using Export/Import.....	9
Import and Use in Your Main File.....	9
Render html elements and forms.....	10
Setup Express.....	10
Render Basic Element on Home Page.....	10
Make and Render a Login Form.....	10
Handle POST Request (Form Submission).....	11
Import HTML Elements From Other Files.....	11
home.js.....	11
Render html file.....	12
Create and render HTML files.....	12
Make node server with express js.....	13
Get Absolute Path.....	13
Now you're ready to open the browser:.....	14
Make 404 Page in Node.js Page not found.....	14
What is a 404 Page?.....	14
Why is 404 Page Important?.....	14
Make a File for 404 Page.....	14

Use Function for 404 Page in Express.....	15
Improvement: Make Common Absolute Path.....	15
Add CSS with Express js.....	15
Add CSS in HTML Page (Inline Method).....	15
Make External CSS File.....	16
Add CSS Link in HTML.....	17
Serve Static Folder in Express.....	17
Middleware in express js.....	18
What is Middleware in Express.js?.....	18
How Middleware Works.....	18
How the Client, Middleware, and Server Work Together.....	19
Middleware Example (code).....	19
Types of Middleware.....	20
Age Check & IP Address Middleware with Code Examples.....	21
Create Server.....	21
Middleware for Age Check.....	21
Function Purpose:.....	21
Middleware for IP Check.....	22
Function Purpose:.....	22
How to Find Your IP Address.....	22
Route Middleware with Code Examples.....	23
What is Route Middleware?.....	23
How to Make Route Middleware.....	23
How to Apply Route Middleware.....	23
Built-In Middleware with Code Examples.....	24
Common Built-In Middleware.....	24
Basic Code.....	24
Form Data Undefined.....	24
Use express.static().....	25
Folder Structure:.....	25
External Middleware in Express.js with Example.....	25
What is External Middleware in Express.js?.....	25
Example: morgan Middleware.....	26
Steps to Use Morgan Middleware.....	26
1. Install morgan.....	26
Use it in your Express app.....	26
Example Output in Terminal.....	26
To Verify Time Logging.....	26
Output in Terminal:.....	27
Error Handling Middleware in Express.js.....	27
What is Error-Handling Middleware?.....	27
Basic Error Handler.....	27
Custom Error with next(error).....	27

Template engine in express JS.....	28
What is a Template Engine?.....	28
Popular Template Engines.....	28
How to Install EJS.....	28
EJS File Structure.....	28
Example: home.ejs.....	29
index.js.....	29
Submit form data and display on ejs template engine page.....	30
Make form with ejs template engine.....	30
Create files in view.....	30
submitUser.ejs.....	30
Make routes.....	31
Get Submitted data.....	31
Display submitted on ejs template page.....	31
Loops and Conditions in EJS Template Files.....	32
Make Route and Pass Array in EJS File.....	32
Create EJS File.....	32
Create an Array and Pass it to EJS.....	32
Apply Loop in EJS File.....	33
Apply Condition in EJS.....	33
MVC Architecture Example with Node.js.....	33
What is MVC?.....	34
MVC Flow Summary.....	34
File Structure in MVC.....	34
Step-by-Step Setup.....	34
Create Basic Express App.....	35
In index.js:.....	35
Create Controller – controllers/usersController.js.....	35
Create Model – models/usersModel.js.....	35
Create View – views/users.ejs.....	35
Dynamic Routes with Example.....	36
What Are Routes?.....	36
What Are Dynamic Routes?.....	37
Creating a Dynamic User Profile Page.....	37
Create a Dynamic Route to Handle User Profile.....	37
How It Works:.....	38
API Example with Dynamic Routes.....	38
Create a JSON File with User Data.....	38
Set Up Express Server.....	39
Fetch a Single User by ID (Dynamic Route).....	40
Fetch a User by Name (Case-Insensitive Match).....	40

Express JS

What is Express.js?

Express.js is a fast, minimal, and flexible web application framework built on **Node.js**. It helps you easily create web servers, APIs, and routes.

Why Use Express.js?

Simple Routing

Middleware Support

Template Engines

REST API Ready

Lightweight and Fast

Easy to Learn

How to Set Up Express.js

Create a New Project Folder

Express-2025

Initialize Node.js Project

```
npm init
```

Fill the details like:

Package Name: **node-js-2025**

Description: **node js and express js tutorials in hindi 2025**

Entry Point: **index.js** (default)

Install Express

```
npm install express --save
```

Create Entry File – **index.js**

Basic Express Server

```
const express = require("express");

const app = express();

app.get("", (req, resp) => {
  resp.send("<h1>Basic Node js example</h1>");
});

app.get("/about", (req, resp) => {
  resp.send("<h1>This is about page</h1>");
});

app.get("/contact", (req, resp) => {
  resp.send("<h1>This is contact page</h1>");
});

app.listen(3200);
```

Explanation:

- `app.get()` – Handles GET requests.
- `resp.send()` – Sends HTML or plain text response.
- `app.listen(3200)` – Server listens on port 3200.

Auto Restart with Nodemon

To avoid restarting the server manually every time:

Install Nodemon

```
npm i nodemon --save-dev
```

Then run your project using:

```
npx nodemon index.js
```

Now your server will auto-refresh on every file save.

How Express JS Works Explained

Understand Express.js Flow

Require Express

```
const express = require("express");
```

We use `require("express")` to include the Express module.

Why we can't use Express directly?

Because Express returns a function. To use it, we need to call it:

```
const app = express();
```

Alternatively, you can do it like this in one line:

```
const app = require("express")();
```

This creates an Express application and stores it in the `app` variable.

Start the Server

```
app.listen(3200);
```

This tells Express to start a server on port `3200`.

Check app Object

If you want to inspect what's inside the Express `app`, use:

```
console.log(app);
```

This will show all the available methods and properties of the Express app.

Express.js Routes and Flow

Basic Route Example

```
app.get("", (req, resp) => {  
  resp.send("<h1>Home Page</h1>");  
});
```

`.get()` is used to handle **GET** requests.

It takes **two parameters**: a path ("" or `/about`) and a callback function.

Multiple Routes Example

```
app.get("", (req, resp) => {  
  resp.send("<h1>Home Page</h1>");  
});  
  
app.get("about", (req, resp) => {  
  resp.send("<h1>About Page</h1>");  
});  
  
app.get("/", (req, resp) => {  
  resp.send("<h1>Home Page 2</h1>");  
});
```

Note:

Express matches and executes the **first matching route**. So if "" and `/` look similar, only the first one is executed.

Module Type in package.json Explained

Why ES Modules?

By default, Node.js uses **CommonJS** syntax (`require`, `module.exports`).

If you try:

```
import express from 'express';
```

You'll get an error, because Node doesn't recognize `import/export` without proper setup.

Enable ES Module Support

In your `package.json`, add this line:

```
"type": "module"
```

This tells Node.js to interpret your files using **ES module syntax**.

Before (CommonJS Syntax)

```
const express = require("express");
```

After (ES Module Syntax)

```
import express from 'express';  
const app = express();
```

Create Pages Using Export/Import

Create `home.js`

`pages/home.js`

```
export default function home() {  
  return `<h1>Home Page with ES export way</h1>`;  
}
```

Create `about.js`

File: `pages/about.js`

```
export default function about() {  
  return `<h1>About Page</h1>`;  
}
```

Create `contact.js`

`pages/contact.js`

```
export default function contact() {  
  return `<h1>Contact Page</h1>`;  
}
```

Import and Use in Your Main File

File: `index.js`

```
import express from 'express';  
import home from './pages/home.js';  
import about from './pages/about.js';  
import contact from './pages/contact.js';
```



```
const app = express();

app.get("/", (req, res) => {
  res.send(home());
});

app.get("/about", (req, res) => {
  res.send(about());
});

app.get("/contact", (req, res) => {
  res.send(contact());
});

app.listen(3200);
```

Render html elements and forms

Setup Express

Start by importing **express** and creating an **app** instance:

```
import express from "express";
const app = express();
```

Render Basic Element on Home Page

```
app.get("/", (req, resp) => {
  resp.send("<h1>Home Page</h1>");
});
```

This will render a simple **Home Page** when you visit <http://localhost:3200/>.

Make and Render a Login Form

```
app.get("/login", (req, resp) => {
  resp.send(`<form action="/submit" method="post">
    <input type="text" placeholder="name">
    <br/>
```

```

        <br/>
        <input type="password" placeholder="Enter Password">
        <br/>
        <br/>
        <button>Login</button>
      </form>`);
    });
  });
});

```

Visit <http://localhost:3200/login> to view a basic login form.

Handle POST Request (Form Submission)

```

app.post("/submit", (req, resp) => {
  resp.send(`<h1>Data Submitted</h1>`);
});

```

When the form is submitted, this route handles the POST request and returns a success message.

Import HTML Elements From Other Files

Organize our HTML into separate files and import them.

home.js

```

export default function home() {
  return "<h1>Home Page</h1> <a href='/login'>Go to Login</a>";
}

```

login.js

```

export default function login() {
  return `<form action="/submit" method="post">
    <input type="text" placeholder="name">
    <br/>
    <br/>
    <input type="password" placeholder="Enter Password">
    <br/>
    <br/>
    <button>Login</button>
    <a href="/">Go to Home</a>
  </form>`;
}

```

submit.js

```
export default function submit() {  
  return "<h1>Data Submitted</h1> <a href='/'>Go to Home</a>";  
}
```

Final Code — [index.js](#)

```
import home from "./pages/home.js";  
import login from "./pages/login.js";  
import submit from "./pages/submit.js";  
  
const app = express();  
  
app.get("/", (req, resp) => {  
  resp.send(home());  
});  
app.get("/login", (req, resp) => {  
  resp.send(login());  
});  
  
app.post("/submit", (req, resp) => {  
  resp.send(submit());  
});  
  
app.listen(3200, () => {  
  console.log("Server is running on port 3200");  
});
```

Render html file

Create and render HTML files

home.html

```
<h1>Home Page</h1>  
<a href="login">Go to Login Page</a>
```

login.html

```
<h1>Login Page</h1>
<a href="/">Go to Home Page</a>
```

about.html

```
<h1>About Page</h1>
<a href="/">Go to Home Page</a>
```

Make node server with express js

```
import express from "express";

const app = express();

app.get("/", (req, resp) => {
  resp.sendFile("view/home.html");
});

app.listen(3200);
resp.sendFile("view/home.html");
```

This gives you an error because you need an **absolute path**.

Get Absolute Path

Use the **path** module to get absolute path.

```
import express from "express";
import path from "path";

const app = express();

app.get("/", (req, resp) => {
  const absPath = path.resolve("view/home.html");
  resp.sendFile(absPath);
});

app.get("/login", (req, resp) => {
  const absPath = path.resolve("view/login.html");
  resp.sendFile(absPath);
});
```

```
app.get("/about", (req, resp) => {  
  const absPath = path.resolve("view/about.html");  
  resp.sendFile(absPath);  
});  
app.listen(3200);
```

We used `path.resolve()` to get the absolute path of HTML files.

Now you're ready to open the browser:

Home → <http://localhost:3200/>

Login → <http://localhost:3200/login>

About → <http://localhost:3200/about>

Make 404 Page in Node.js Page not found

What is a 404 Page?

A **404 page** means "**Page Not Found**". This shows up when a user tries to visit a page on your website that doesn't actually exist.

Why is 404 Page Important?

Without a 404 page:

- User sees a blank or error page.

- They may think your website is broken or not working.

With a 404 page:

- You can show a friendly message.

- Tell the user that the website is fine, but the page they requested doesn't exist.

Make a File for 404 Page

Create a new HTML file named `404.html` inside the `view` folder.

You can use ChatGPT to generate a beautiful and simple 404 HTML template.

Use Function for 404 Page in Express

Express provides a special function `app.use()` — it's a **multi-purpose middleware** and is also used to handle 404 errors.

```
app.use((req, resp) => {  
  const absPath = path.resolve("view/404.html");  
  resp.status(404).sendFile(absPath);  
});
```

Improvement: Make Common Absolute Path

To avoid repeating `path.resolve()` again and again, create a **common absolute path** for the whole folder (not individual files):

```
const absPath = path.resolve("view");
```

Then use it like this:

```
app.use((req, resp) => {  
  resp.status(404).sendFile(absPath + "/404.html");  
});
```

Add CSS with Express js

Add CSS in HTML Page (Inline Method)

```
<!DOCTYPE html>  
<html lang="en">  
  <head>  
    <meta charset="UTF-8" />  
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />  
    <title>Home Page</title>  
    <style>  
      h1 {  
        color: orange;  
      }  
  
      a {  
        text-decoration: none;  
        padding: 10px 20px;
```

```
background: blue;
color: white;
border-radius: 10px;
}

a:hover {
background: rgb(86, 86, 220);
}
</style>
</head>
<body>
<h1>Home Page</h1>
<a href="login">Go to Login Page</a>
</body>
</html>
```

Is file mein `<style>` tag ke andar hi CSS likha gaya hai. Yeh basic styling test karne ke liye kaafi hai.

Make External CSS File

Create a file:

`public/css/style.css`

style.css

```
h1 {
color: orange;
}

a {
text-decoration: none;
padding: 10px 20px;
background: blue;
color: white;
border-radius: 10px;
}
```

```
a:hover {  
  background: rgb(86, 86, 220);  
}
```

Add CSS Link in HTML

In `home.html` file head section, change:

```
<link rel="stylesheet" href="css/style.css" />
```

❌ Don't use:

```
<link rel="stylesheet" href="public/css/style.css" />
```

Serve Static Folder in Express

```
import express from "express";  
import path from "path";  
  
const app = express();  
  
const publicPath = path.resolve("public");  
  
app.use(express.static(publicPath));
```

The `express.static()` function allows you to make the files inside the `public` folder accessible to the browser.

So when your HTML file includes a line like:

```
<link rel="stylesheet" href="css/style.css" />
```

...the browser will be able to **find and load** that CSS file from the `public/css` folder, **because** you've told Express to serve everything in the `public` folder using:

```
app.use(express.static(publicPath));
```

Without this, the browser wouldn't be able to locate the CSS file, even if it's in your project folder. This function tells Express:

Middleware in express js

What is Middleware in Express.js?

Middleware in Express.js is a **function** that gets executed **before** the final route handler.

It can be used to check requests, log activities, authenticate users, validate data, and more.

How Middleware Works

```
function middleware(req, res, next) {  
  // Do something with req or res  
  next(); // Pass control to the next middleware or route  
}
```

req is the request object

res is the response object

next() moves to the next middleware or route handler

How the Client, Middleware, and Server Work Together

Client → Sends HTTP request to server (e.g., GET, POST)

Middleware → Intercepts and processes the request

Server → Executes the final route handler and sends a response

Middleware Example (code)

```
import express from "express";  
  
const app = express();  
  
function checkRoute(req, res, next) {  
  console.log("user is accessing " + req.url + " Page");  
  next();  
}  
  
app.use(checkRoute);  
  
app.get("/", (req, res) => {
```

```
resp.send("Home Page");
});

app.get("/users", (req, resp) => {
  resp.send("users Page");
});

app.get("/products", (req, resp) => {
  resp.send("Products Page");
});

app.listen(3200);
```

This middleware (**checkRoute**) runs for **every route** and logs the page being accessed before the actual route is handled.

You can also write it like this using an arrow function:

```
app.use((req, resp, next) => {
  console.log("user is accessing " + req.url + " Page");
  next();
});
```

Types of Middleware

Application-level Middleware

Registered using: **app.use()** or **app.get()**

Purpose: Used to apply middleware globally or to specific routes in the main app.

Example Use: Logging, request parsing, etc.

Router-level Middleware

Registered using: **router.use()**

Purpose: Works like application-level middleware, but applies to route instances (e.g., for grouping **/admin** or **/user** routes).

Example Use: Authorization checks in a specific route group.

Built-in Middleware

Comes with Express.js.

Common example: `express.static()` to serve static files (like CSS, images).

Purpose: Provides ready-to-use functionality without extra installation.

Error-handling Middleware

Defined with four arguments: `(err, req, res, next)`

Purpose: Catches and handles errors across the application.

Example Use: Sending custom error messages, logging errors.

Third-party Middleware

Installed via npm, and used with `require()` or `import`.

Purpose: Add advanced functionalities.

Common examples:

`morgan` (logging)

`cors` (Cross-Origin Resource Sharing)

`body-parser` (parsing form data)

Age Check & IP Address Middleware with Code Examples

Create Server

```
import express from "express";

const app = express();

app.get("/", (req, res) => {
  res.send("<h1>Home Page</h1>");
});

app.get("/login", (req, res) => {
  res.send("<h1>Login Page</h1>");
});

app.get("/admin", (req, res) => {
  res.send("<h1>Admin Page</h1>");
});

app.listen(3200);
```

Middleware for Age Check

Function Purpose:

Checks if user provided **age** in the query (**?age=20**)

Blocks access if age is missing or less than 18

Code:

```
function ageCheck(req, resp, next) {  
  if (!req.query.age || req.query.age < 18) {  
    resp.send("Alert ! You can not access this page");  
  } else {  
    next();  
  }  
}  
  
app.use(ageCheck); // Applies globally to all routes
```

Middleware for IP Check

Function Purpose:

Logs the visitor's IP address

Blocks access for specific IP (example: **192.168.1.23**)

```
function ipCheck(req, resp, next) {  
  const ip = req.socket.remoteAddress;  
  console.log(ip); // View in terminal  
  
  if (ip.includes("192.168.1.23")) {  
    resp.send("Alert ! You can not access this page");  
  } else {  
    next();  
  }  
}  
  
app.use(ipCheck); // Applies globally
```

How to Find Your IP Address

Windows Command Prompt:

```
ipconfig
```

Mac Terminal:

```
ipconfig getifaddr en0
```

Route Middleware with Code Examples

What is Route Middleware?

Middleware applied to **specific routes only**

Executes **before the final route handler**

How to Make Route Middleware

```
function checkAgeRouteMiddleware(req, resp, next) {  
  console.log(req.query);  
  
  if (!req.query.age || req.query.age < 18) {  
    resp.send("Yoru are not allowed to use this website");  
  } else {  
    next();  
  }  
}  
  
function checkUrl(req, resp, next) {  
  console.log("this req url is " + req.url);  
  next();  
}
```

How to Apply Route Middleware

Use middleware as a **second parameter**

You can apply **multiple middleware functions** on a single route

```
app.get("/", (req, resp) => {  
  resp.send("<h1>Home page</h1>");  
});
```

```
});

app.get("/login", checkUrl, (req, resp) => {
  resp.send("<h1>login page</h1>");
});

app.get("/users", checkAgeRouteMiddleware, checkUrl, (req, resp) => {
  resp.send("<h1>users page</h1>");
});

app.get("/products", checkAgeRouteMiddleware, checkUrl, (req, resp) => {
  resp.send("<h1>products page</h1>");
});
```

Built-In Middleware with Code Examples

Built-in middleware is already provided by Express. You **don't need to create** it — you just **use it directly**.

Common Built-In Middleware

`express.static()`

- Serves static files like CSS, JS, images.
- Example: Serves files from a folder (`public`).

`express.urlencoded()`

- Parses incoming form data (`application/x-www-form-urlencoded`).
- Helps you access form data via `req.body`.

Basic Code

```
import express from "express";
const app = express();
```

Form Data Undefined

```
app.post("/submit", (req, resp) => {  
  console.log("user login details are :", req.body); // undefined without middleware  
  resp.send("Submit Page");  
});
```

Use built-in middleware before routes:

```
app.use(express.urlencoded({ extended: false }));
```

This allows Express to read data from the form (like **email** & **password**).

Use `express.static()`

To serve static CSS file:

Create folder: **public**

Add file: **public/style.css**

In HTML:

```
<link rel="stylesheet" href="/style.css" />
```

In index.js:

```
app.use(express.static("public"));
```

Now Express will automatically serve CSS and other static files from **public/**.

Folder Structure:

```
project/  
├── public/  
│   └── style.css  
├── view/  
│   └── home.html  
└── server.js
```

External Middleware in Express.js with Example

What is External Middleware in Express.js?

External middleware is **not built into Express.js**.

You must **install** it separately via npm.

It adds **extra functionality** (like logging, CORS, authentication, etc.)

Example: **Morgan** Middleware

morgan is an external middleware used for **logging HTTP requests** in the terminal.

Helpful for **debugging** and **monitoring** incoming requests.

Steps to Use Morgan Middleware

1. Install morgan

```
npm i morgan
```

Import it

```
import morgan from "morgan";
```

Use it in your Express app

```
app.use(morgan("dev"));
```

"dev" is a predefined format that logs method, URL, status code, response time, etc.

Example Output in Terminal

```
GET / 200 2.012 ms - 9
GET /login 200 1.223 ms - 12
GET /random 404 3.314 ms - 142
```

To Verify Time Logging

Add a route with a **delay**:

```
app.get("/wait", (req, resp) => {
  setTimeout(() => {
    resp.send("result after 1 sec");
  }, 1000);
});
```

Now open:

<http://localhost:3200/wait>

Output in Terminal:

```
GET /wait 200 1011.997 ms - 18
```


Shows accurate request timing.

Error Handling Middleware in Express.js

What is Error-Handling Middleware?

It is a **special type of middleware** in Express.js.

It catches and handles **errors** that occur during route processing.

Helps to show a **friendly message** instead of crashing the server.

Basic Error Handler

```
app.get("/users", (req, resp) => {  
  resp.send1("User Page"); // Typo will cause error  
});  
  
function errorHandler(error, req, resp, next) {  
  resp.status(error.status || 500).send("Try after some time");  
}  
  
app.use(errorHandler);
```

Output: Server doesn't crash, shows:

"Try after some time"

Custom Error with `next(error)`

```
app.get("/error", (req, resp, next) => {  
  const error = new Error("Something went wrong");  
  error.status = 404;  
  next(error); // Pass error to middleware  
});  
  
app.use((error, req, resp, next) => {  
  resp.status(error.status || 500).send("Try after some time");  
});
```

Output: Returns status 404 with message

"Try after some time"

Template engine in express JS

What is a Template Engine?

A **template engine** lets you **generate dynamic HTML** using **server-side data**.

Helps merge backend data with frontend templates (like variables in HTML).

Popular Template Engines

EJS (Embedded JavaScript) ✓

Pug (formerly Jade)

Handlebars

How to Install EJS

```
npm i ejs
```

EJS File Structure

```
project/  
|  
|— views/  
|   |— home.ejs  
|— public/  
|   |— style.css  
|— index.js
```

Note: EJS automatically looks inside the **views/** folder

Example: **home.ejs**

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8" />  
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />  
  <title>Home Page</title>  
  <link rel="stylesheet" href="style.css" />  
</head>  
<body>  
  <h1>Name <%= name %></h1>  
  <h2>Yt Channel: <%= ytChannel %></h2>
```

```
<h2>Age : <%= age %></h2>
</body>
</html>
```

<%= ... %> is used to embed JavaScript values in HTML

index.js

```
import express from "express";
const app = express();

app.set("view engine", "ejs");

app.get("/", (req, resp) => {
  resp.render("home", {
    name: "Anil Sidhu",
    ytChannel: "Code Step By Step",
    age: 29,
  });
});

app.listen(3200);
```

app.set("view engine", "ejs"); tells Express to use EJS

resp.render("home", {...}) sends data to the EJS file

Submit form data and display on ejs template engine page

Make form with ejs template engine

Create files in view

addUser.ejs

```
<form action="/submit-user" method="post">
  <input type="text" placeholder="enter user name" name="name" />
  <br /><br />
  <input type="text" placeholder="enter user age" name="age" />
  <br /><br />
  <input type="text" placeholder="enter user email" name="email" />
```

```
<br /><br />
<button>Add User</button>
</form>
```

This form sends data using **POST** method to the route **/submit-user**.

submitUser.ejs

```
<h3>Name: <%= name %></h3>
<h3>age:<%= age %></h3>
<h3>Email:<%= email %></h3>
```

Make routes

Now, connect the frontend form and result page using routes in your Express server.

```
app.get("/add-user", (req, resp) => {
  resp.render("addUser");
});

app.post("/submit-user", (req, resp) => {
  console.log(req.body);
  resp.render("submitUser", req.body);
});
```

app.get loads the form

app.post receives form data and sends it to the template

Get Submitted data

Use **req.body** to receive the data that the user submitted via the form.

Important: To make this work, we must add this middleware:

```
app.use(express.urlencoded({ extended: false }));
```

Display submitted on ejs template page

In **submitUser.ejs**, we already used EJS tags to display values:

```
<h3>Name: <%= name %></h3>
<h3>age:<%= age %></h3>
```

```
<h3>Email:<%= email %></h3>
```

These values come from the `req.body` we passed in the POST route.

Loops and Conditions in EJS Template Files

Make Route and Pass Array in EJS File

In your `index.js` file (or main Express file), first set up a route that will render your EJS file.

```
app.get("/users", (req, resp) => {  
  resp.render("users"); // This will render 'users.ejs' file  
});
```

Note: EJS by default looks inside the `views` folder, so make sure your EJS file is there.

Create EJS File

Inside the `views` folder, create a new file called `users.ejs`.

```
<!DOCTYPE html>  
<html lang="en">  
  <head>  
    <meta charset="UTF-8" />  
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />  
    <title>Users List</title>  
  </head>  
  <body>  
    <h1>User List Page</h1>  
  </body>  
</html>
```

Create an Array and Pass it to EJS

Now go back to your route and pass a list of users to the template.

```
const users = ["anil", "sidhu", "sam", "peter", "bruce"];  
  
app.get("/users", (req, resp) => {  
  resp.render("users", { users: users });  
});
```

Here, we are passing the `users` array to the `users.ejs` file. You can now use this data inside the template.

Apply Loop in EJS File

Now, inside `users.ejs`, use a `for` loop to show each user in a list:

```
<ul>
<% for (let i = 0; i < users.length; i++) { %>
  <li><%= users[i] %></li>
<% } %>
</ul>
```

Use `<% %>` to write JavaScript (like loops and conditions).

Use `<%= %>` to print values to HTML.

Don't use `=` if you're not printing anything.

Apply Condition in EJS

Let's say we want to show a message only if the user is logged in. First, create a variable in your route:

```
app.get("/users", (req, resp) => {
  const isLogin = true;
  resp.render("users", { users: users, isLogin });
});
```

Now use this variable in your EJS file with an `if` condition:

```
<% if (isLogin) { %>
<h2>User is Logged In</h2>
<% } %>
```

MVC Architecture Example with Node.js

When working on real-world Node.js projects, it's important to organize your code properly. One of the most common and effective patterns for doing this is the **MVC Architecture**.

What is MVC?

MVC stands for:

M – Model (Handles the data)

V – View (Handles the UI)

C – Controller (Handles the logic and routes)

MVC Flow Summary

Here's how everything connects:

User → Controller → Model → View → User

The **User** interacts with the app.

The **Controller** receives the request and communicates with the **Model**.

The **Model** processes data and sends it back.

The **View** displays the result to the user.

File Structure in MVC

Create the following folders in your project:

```
project/
├── controllers/
│   └── usersController.js
├── models/
│   └── userModel.js
├── views/
│   └── users.ejs
└── index.js
```

Step-by-Step Setup

Create Basic Express App

In **index.js**:

```
import express from "express";
import { handleUsers } from "../controllers/usersController.js";

const app = express();
app.set("view engine", "ejs");

// Route
```

```
app.get("/user", handleUsers);

app.listen(3200, () => {
  console.log("Server is running on port 3200");
});
```

Create Controller – [controllers/usersController.js](#)

The controller connects the model and view.

```
import { userList } from "../models/usersModel.js";

export function handleUsers(req, resp) {
  const usersData = userList(); // Get data from model
  resp.render("users", { users: usersData }); // Send data to view
}
```

Create Model – [models/usersModel.js](#)

```
export function userList() {
  return ["Anil", "Sidhu", "Sam", "Peter", "Bruce"];
}
```

Create View – [views/users.ejs](#)

This is what the user sees on the browser.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Users List</title>
</head>
<body>
  <h1>User List Page</h1>
  <ul>
    <% for (let i = 0; i < users.length; i++) { %>
      <li><%= users[i] %></li>
    <% } %>
  </ul>
```



```
</body>
</html>
```

We loop through the users array and display each user in a list.

Dynamic Routes with Example

In real-world applications, we often need to create web pages that respond differently based on the **data passed in the URL**. That's exactly where **dynamic routes** come in!

What Are Routes?

In a web app, when a user visits a specific **URL**, we use a **route** to handle that request.

Example:

```
app.get("/about", (req, resp) => {
  resp.send("About Page");
});
```

This route handles requests made to localhost:3200/about.

What Are Dynamic Routes?

Dynamic routes allow us to pass dynamic data **through the URL** using parameters.

Instead of writing a new route for every user, we write one route that accepts any user name as a **parameter**.

Creating a Dynamic User Profile Page

Create a List of Users with Profile Links

```
app.get("/", (req, resp) => {
  const users = ["anil", "sam", "peter", "sidhu"];
  let data = `<ul>`;

  for (let i = 0; i < users.length; i++) {
    data += `<li><a href="/user/${users[i]}">${users[i]}</a></li>`;
  }
}
```

```
data += `</ul>`;
resp.send(data);
});
```

This will render a list of user names as links like:

```
/user/anil
/user/sam
```

Create a Dynamic Route to Handle User Profile

```
app.get("/user/:name", (req, resp) => {
  const userName = req.params.name;
  resp.send(`This is ${userName}'s Profile Page`);
});
```

Here, `:name` is a dynamic parameter.

`req.params.name` captures whatever is passed in the URL.

How It Works:

Visiting `/user/anil` will display:

"This is anil's Profile Page"

Visiting `/user/sidhu` will display:

"This is sidhu's Profile Page"

This is super useful for applications like:

Blog posts (`/post/:id`)

User profiles (`/user/:username`)

Product pages (`/product/:slug`)

API Example with Dynamic Routes

Create a JSON File with User Data

We'll start by storing our user data in a separate JSON file. Create a folder called `data` and inside it, create a file named:

`user.json`

```
[
{
```

```
"id": 1,
"name": "Anil",
"age": 30,
"email": "anil@test.com"
},
{
  "id": 2,
  "name": "Sam",
  "age": 32,
  "email": "sam@test.com"
},
{
  "id": 3,
  "name": "Peter",
  "age": 20,
  "email": "peter@test.com"
},
{
  "id": 4,
  "name": "Bruce",
  "age": 18,
  "email": "bruce@test.com"
}
]
```

Set Up Express Server

Now, create an Express server in your `index.js` or `app.js` file:

```
import express from "express";
import userData from "../data/user.json" assert { type: "json" };

const app = express();
const PORT = 3200;

// Default route to send full user list
app.get("/", (req, resp) => {
  resp.send(userData);
});
```

```
});  
  
app.listen(PORT, () => {  
  console.log(`Server running on http://localhost:${PORT}`);  
});
```

Fetch a Single User by ID (Dynamic Route)

Create a dynamic route to fetch user data using the ID passed in the URL:

```
app.get("/user/:id", (req, resp) => {  
  const id = req.params.id;  
  const filteredData = userData.filter((user) => user.id == id);  
  resp.send(filteredData);  
});
```

Explanation:

`:id` is a dynamic parameter.

`req.params.id` captures the ID from the URL.

We use `filter()` to match the ID in the JSON data.

GET `/user/2` → returns data of Sam.

Fetch a User by Name (Case-Insensitive Match)

You can also get users by their name:

```
app.get("/user-by-name/:name", (req, resp) => {  
  const name = req.params.name;  
  const filteredData = userData.filter(  
    (user) => user.name.toLowerCase() === name.toLowerCase()  
  );  
  resp.send(filteredData);  
});
```

Example:

GET `/user-by-name/peter` → returns Peter's details.